

Kịch bản video demo — HW06 · 23127195

MSSV: 23127195 · **Bài tập:** HW06 — API Testing (SUT: EShop)
Thời lượng đề xuất: 6–8 phút · **Ngôn ngữ:** tiếng Việt, tự thuyết minh
Nộp: link YouTube (có thể để *Unlisted*)

Trọng tâm theo §7 của đề bài: video "*showing it generate tests for one API*" — tức là phần chính phải là **Agent Skill sinh test case**, không phải phần chạy Postman.
Chọn **API-2 (FR-09 — mã giảm giá)** để demo vì có bảng quyết định 5 điều kiện và oracle số học rõ ràng, dễ nhìn thấy giá trị của bộ sinh.

Chuẩn bị trước khi quay

```
# 1. Khởi động SUT
cd ../../.sut/eshop-sut/backend && node server.js

# 2. Mở sẵn (mỗi thứ một tab / cửa sổ):
#   - Terminal ở thư mục 23127195/
#   - Postman desktop, đã import collection API-2 + environment
#   - Postman Console (Ctrl+Alt+C)
#   - Trình duyệt: agent-skill/DESIGN.md, sơ đồ tự vẽ, bugs/BUG_REPORTS.md
```

- [] Phóng to cỡ chữ terminal (ít nhất 16pt) để người xem đọc được
- [] Đóng các tab/thông báo không liên quan
- [] Thử micro trước

Phân đoạn

0:00 – 0:40 · Danh tính và tổng quan

Làm: chạy trong terminal

```
whoami && hostname
git log --oneline -8
```

Nói:

"Em là <họ tên>, MSSV 23127195. Bài HW06 kiểm thử ba API của hệ thống EShop: FR-04 hồ sơ cá nhân thuộc Pool A, FR-09 mã giảm giá thuộc Pool B, và FR-16 import sản phẩm

thuộc Pool C. Tổng cộng 144 test case, tìm ra 24 lỗi trong đó có 4 lỗi mức Critical."

0:40 – 2:30 · Agent Skill sinh test case ★ phần chính

a) Giải thích thiết kế (mở sơ đồ tự vẽ)

Nói:

"Ý tưởng trung tâm của bộ sinh là: chỉ dùng mô hình ngôn ngữ cho đúng một việc — đọc hiểu tài liệu đặc tả. Đó là giai đoạn G1, biến tài liệu thành một cấu trúc gọi là EndpointModel. Năm giai đoạn còn lại là mã tắt định. Tính đầy đủ đến từ một **danh mục quy tắc phân vùng**, không đến từ mô hình — vì nếu chỉ bảo AI 'sinh test case đi' thì nó sẽ sinh dày ở chỗ dễ và bỏ sót ở chỗ khó."

b) Chạy thật bộ sinh

```
python agent-skill/pseudocode/generator.py --demo
```

Nói khi kết quả hiện ra:

"Từ một EndpointModel, bộ sinh cho ra 44 test case: 26 phân vùng miền, 14 bảo mật, 4 lược đồ — và qua bộ kiểm tra G6a với 0 lỗi. Con số này rất gần với 45 test case mà em làm bằng quy trình đầy đủ cho chính API đó."

c) Chỉ ra một quy tắc cụ thể tạo ra khác biệt

Mở `agent-skill/pseudocode/generator.py`, cuộn tới nhánh `c["loai"] == "bien"`.

Nói:

"Đây là chỗ quan trọng nhất. Với mỗi ràng buộc biên, danh mục bắt buộc sinh đủ **sáu điểm**: min trừ 1, min, min cộng 1, và tương tự ở đầu max. Điểm `v == min` chính là điểm phân biệt một cài đặt dùng dấu `>` với một cài đặt dùng `>=`. Nếu chỉ hỏi AI chung chung thì nó bỏ qua điểm này — và đó đúng là lỗi BUG-A2-03 em tìm được."

2:30 – 4:00 · Từ test case sinh ra đến Postman collection chạy được

a) Biên dịch

```
python postman/scripts/build_collections.py
```

b) Mở Postman — cho thấy collection API-2 với các thư mục theo kỹ thuật kiểm thử:

01 - Decision Table C1, 03 - Decision Table C3 (BVA), 05 - C5 + State Transition, ...

c) Mở pre-request script cấp collection, chỉ vào dòng gắn header:

```
pm.request.headers.upsert({ key: 'X-Student-Id', value: sid });
```

d) Mở Postman Console rồi chạy một request — chỉ rõ dòng log:

```
[X-Student-Id] 23127195 -> POST /api/apply-coupon
```

Nói:

"Mọi request đều mang header X-Student-Id, gắn tự động bằng pre-request script ở cấp collection. Đây là bằng chứng bắt buộc theo mục 11 của đề bài."

4:00 – 5:30 · Thi hành bằng Newman và một lỗi tìm được

a) Chạy

```
bash scripts/run_newman.sh api2
```

b) Khi kết quả hiện ra, chỉ vào `TC-A2-032` FAIL và giải thích:

Nói:

"Đây là lỗi Critical BUG-A2-02. Đặc tả FR-09 ghi công thức giảm giá phần trăm là tổng nhân giá trị chia một trăm. Với đơn 500 nghìn và mã giảm 10%, giảm giá phải là 50 nghìn, khách trả 450 nghìn. Nhưng hệ thống trả về giảm giá **âm 4 triệu 500**, và số tiền phải trả là **5 triệu** — khách dùng mã giảm giá lại phải trả gấp 10 lần giá gốc."

c) Tái hiện bằng curl để chứng minh không phụ thuộc Postman:

```
curl -s -X POST localhost:3000/api/apply-coupon -H 'Content-Type: application/json' \
-H 'X-Student-Id: 23127195' -d '{"code":"SAVE10","total_amount":500000,"user_id":2}'
```

5:30 – 6:30 · Bài học: lỗi của chính bộ test

Nói:

"Phần em muốn nhấn mạnh nhất không phải 24 lỗi của hệ thống, mà là hai lỗi của **chính bộ test** do AI sinh ra.

AI viết assertion bất đồng bộ theo mẫu gọi `pm.sendRequest` rồi đặt `done()` sau `pm.expect`. Khi assertion đạt thì chạy đúng. Nhưng khi assertion **hông**, ngoại lệ được ném trước `done()`, và Newman **âm thầm bỏ qua** test đó — không PASS, không FAIL, biến mất khỏi báo cáo.

Hậu quả là test cho lỗi nghiêm trọng nhất của cả bài — leo quyền lên admin — ban đầu hiện ra như đã pass. Em chỉ phát hiện khi đối chiếu báo cáo Newman với kết quả `curl` thủ công: `curl` cho thấy role đã thành admin, còn Newman thì không có dòng FAIL nào tương ứng.

Dấu hiệu là **sự vắng mặt của một assertion**, không phải một assertion sai. Đó là bài học lớn nhất em rút ra: AI đáng tin ở việc phủ có hệ thống, nhưng không đáng tin ở việc tự kiểm chứng chính nó."

(Mở `ai/AI_CRITIQUE.md` trong lúc nói đoạn này.)

6:30 – 7:00 · CI/CD và kết luận

Làm: mở tab GitHub Actions, cho thấy hai lần chạy (một xanh, một đỏ).

Nói:

"Pipeline được thiết kế hai tầng. Vì hệ thống đang có 24 lỗi nên nếu chạy cả 144 test case và bắt build đỏ thì pipeline sẽ luôn đỏ — và một pipeline luôn đỏ thì không ai còn nhìn nó nữa. Nên tầng một chỉ gồm 92 test case đang đạt và bắt buộc phải xanh — nó phát hiện hồi quy. Tầng hai chạy đầy đủ để theo dõi tiến độ sửa lỗi nhưng không chặn build. Em xin hết, cảm ơn thầy cô đã xem."

Sau khi quay

- [] Xem lại: có đoạn nào lộ thông tin cá nhân không nên công khai?
- [] Kiểm tra mã số **23127195** hiển thị rõ ở ít nhất hai chỗ (terminal và Postman Console)
- [] Tải lên YouTube, đặt **Unlisted**
- [] Dán link vào `docs/00_MAIN_REPORT.md` §4 và vào `README.md`
- [] Ghi link vào đây: <điền link YouTube>